Solution: My Calendar I

Problem Statement

Given a 2D array `nums` of size `N x 2`.

- **nums[i] = [start$_i$, end$_i$]**, where start$_i$ is the starting time of the event and end$_i$ is the ending time of the event.

For each `nums[i]`, determine if a requested booking time conflicts with any existing bookings.

Return a boolean array of size `N`, representing whether the booking can be done in the given time interval.

Examples

1. **Example 1**:
   - Input: nums = [[10, 20], [15, 25], [20, 30]]
   - Expected Output: `[true, false, true]`
   - Justification: The first event is booked successfully. The second event overlaps with the first one and is rejected. The third event starts when the first event ends, so it's booked successfully.
2. **Example 2**:
   - Input: [[5, 10], [10, 15], [5, 15]]
   - Expected Output: `[true, true, false]`
   - Justification: The first and second events are booked without overlap. The third event overlaps with both the first and second, so it's rejected.
3. **Example 3**:
   - Input: [[8, 13],[13, 17], [17, 20]]
   - Expected Output: `[true, true, true]`
   - Justification: All events are booked without any overlap, as each event starts exactly when the previous one ends.

Solution

To solve this problem, we manage event scheduling in a personal calendar using a TreeSet, a sorted data structure, to store events in order of their start times. The core of the solution involves processing an array of events, where each event is checked for potential overlaps with already scheduled events. This is done by comparing each new event with its closest neighbors in the TreeSet – the nearest events before and after it. If the new event doesn't overlap with these neighbors (i.e., it starts after the preceding event ends and ends before the subsequent event begins), it's added to the TreeSet. Otherwise, it's rejected. This approach efficiently handles multiple event bookings in one operation, ensuring that no two events overlap in the calendar.

1. **Initialization:**
   - Create a data structure (`SortedSet` in C#, `TreeSet` in Java, `set` in C++, and a list in Python and JavaScript) to store the booked intervals. This data structure will help maintain the intervals in a sorted order based on their start times.
2. **Booking Process:**
   - When a new booking request comes in (represented by a start and end time), we need to determine if it overlaps with any existing bookings.
   - Check for two conditions:
     - The new booking starts after or exactly when the previous booking ends.
     - The new booking ends on or before the next booking starts.
   - If both conditions are met, the booking does not overlap with existing bookings, and we can add it to our data structure.
3. **Insertion:**
   - If the booking is successful (i.e., no overlap), add the new interval to the data structure.
4. **Returning Results:**
   - The function should return a boolean value indicating whether the booking was successful (`true`) or not (`false`).

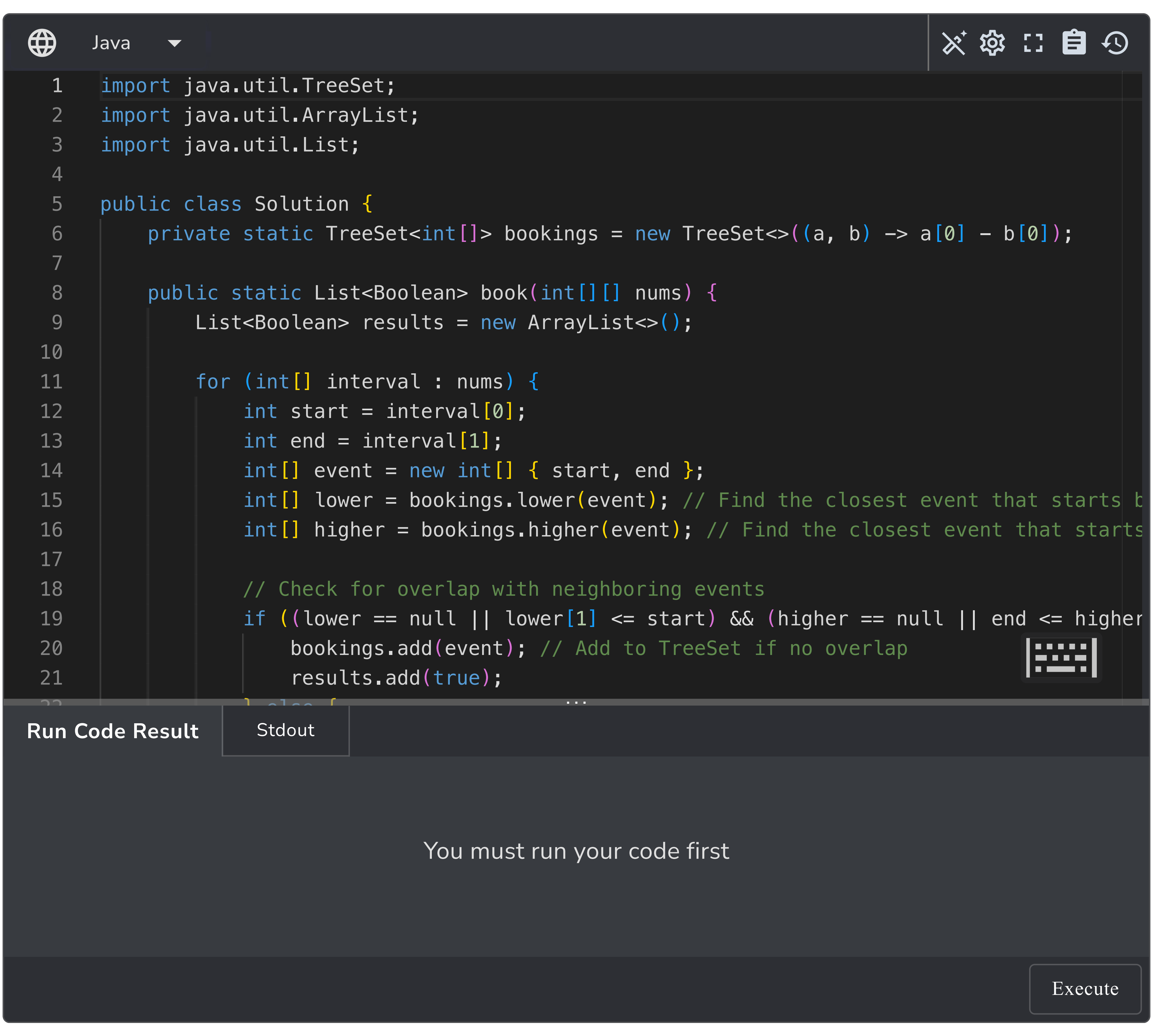
Example Walkthrough

Let's consider the example where we have the following booking requests: [[10, 20], [15, 25], [20, 30]].

1. **Booking `[10, 20]`:**
   - The booking list is empty, so this interval is added without any overlap. The list now contains `[10, 20]`.
   - **Result:** `true`
2. **Booking `[15, 25]`:**
   - We check this against the existing booking `[10, 20]`. The new booking starts before the existing one ends (15 < 20), so this is an overlap.
   - **Result:** `false`
3. **Booking `[20, 30]`:**
   - We check this against the existing booking `[10, 20]`. The new booking starts exactly when the existing one ends (20 == 20), so there is no overlap.
   - The booking `[20, 30]` is added to the list. The list now contains `[10, 20]` and `[20, 30]`.
   - **Result:** `true`

So, the final results of these booking attempts are `[true, false, true]`.

Code



```java
import java.util.TreeSet;
import java.util.ArrayList;
import java.util.List;

public class Solution {
    private static TreeSet<int[]> bookings = new TreeSet<>((a, b) -> a[0] - b[0]);

    public static List<Boolean> book(int[][] nums) {
        List<Boolean> results = new ArrayList<>();

        for (int[] interval : nums) {
            int start = interval[0];
            int end = interval[1];
            int[] event = new int[] { start, end };
            int[] lower = bookings.lower(event); // Find the closest event that starts b
            int[] higher = bookings.higher(event); // Find the closest event that starts

            // Check for overlap with neighboring events
            if ((lower == null || lower[1] <= start) && (higher == null || end <= higher
                bookings.add(event); // Add to TreeSet if no overlap
                results.add(true);
```

You must run your code first

Complexity Analysis

- **Time Complexity:** The primary operations are searching and inserting in the ordered set. Both operations have an average time complexity of O(log n), where n is the number of events in the set.
- **Space Complexity:** The space complexity is O(n), as we need to store each event in the set.

← Previous
My Calendar I (Medium)

Next →
Longest Continuous Subarray (Medium)

☐ Mark as Completed